SITE AUTOMATION

Service watch

AAP 2.7 + ACT (GenAI): find why a container went down, record it, and fix it after approval

Step-by-step setup and test | site-automation 1.43.0 | October 2026

What this is. Step-by-step instructions to have Ansible Automation Platform watch the `stigman` and `nginx` containers. When one is down, ACT (GenAI) finds the root cause, the incident is recorded, and the fix waits for a person to approve it (or, in self-heal mode, is applied at once). It covers the ACT script, the playbooks, the AAP templates, the workflow, approving a fix, and the test: stop a container and watch it come back.

Source
Rendered from `docs/SERVICE_WATCH_DEMO.md` in the site-automation repository. For copy-paste of YAML, use the repository file rather than this PDF.

Host names and addresses in this guide are placeholders. Copy YAML from the repository files, not from the PDF.

Contents

Overview 3

Part 1. The pieces 4

Part 2. The ACT script 4

- What happens on the host during a run . . . 4
- What ACT can and cannot do . . . 5
- What is sent to GenAI . . . 5
- Test ACT by hand first (10 minutes, on the STIG Manager host) . . . 5
- Which containers are watched (same host) . . . 6

Part 3. The playbooks 7

Part 4. Put the code where AAP can reach it 8

Part 5. AAP setup 8

- Step 1. Credential type "ACT model key" . . . 8
- Step 2. Credentials . . . 9
- Step 3. Project . . . 9
- Step 4. Inventory . . . 9
- Step 5. Job templates . . . 10

Part 6. The workflow 10

Part 7. Approving a fix 11

- Who can approve . . . 11
- How to approve . . . 11
- Rehearse first: approve without changing anything . . . 11
- What happens next . . . 11
- When to deny . . . 11

Part 8. The test: stop a container, approve the fix 12

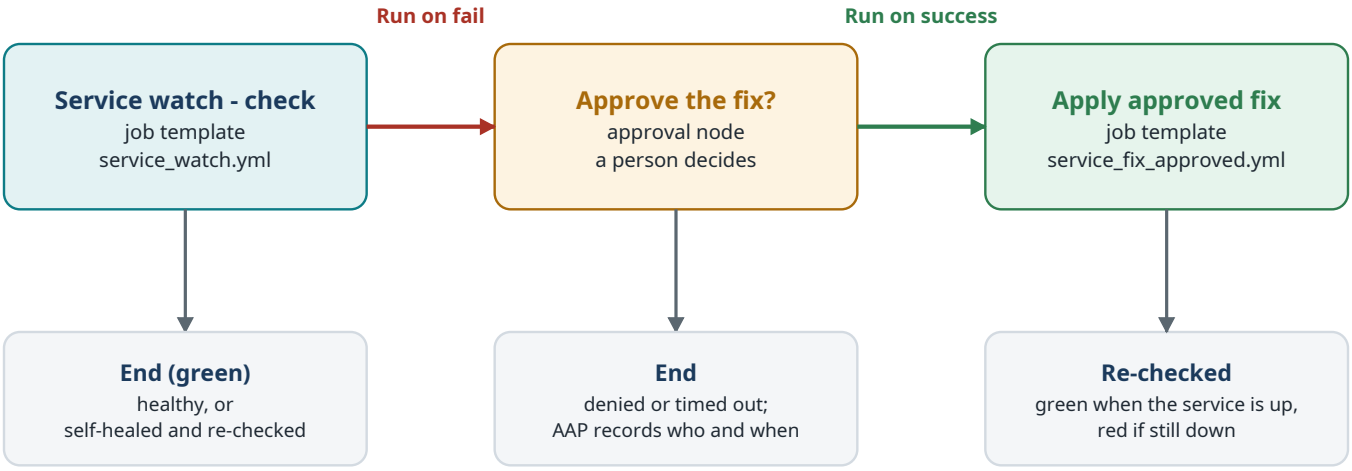
Part 9. If something goes wrong 13

Overview

What it does. AAP checks every podman container on the host that should be running - root's (rootful podman) and each user's (rootless podman) - and finds them by itself, plus any you list (`watch_containers`). When one is down, ACT (using GenAI) finds out why and the incident is recorded. Then, depending on the mode you pick when you launch it:

- **approval** (the default): ACT reports the root cause and proposes the fix, AAP waits for a person to approve it, then applies exactly that fix and checks the container is back;
- **self-heal**: ACT starts the container itself, and the playbook checks it is back.

The test in this guide: stop `nginx` (or `stigman`) by hand, run the workflow, read ACT's root cause and proposed fix, approve it, and watch the container come back.



The workflow. The check job fails on purpose only when a fix needs a person, so an approval request appears only then.

Menu names
The screens are described with the AAP 2.7 wording. Not every click was checked in a live 2.7; if a label differs, follow its meaning.

Part 1. The pieces

File	What it is	Do you edit it?
`vendor/act/act`	**The ACT script**: one Python file. The playbook copies it to the host on every run.	No
`vendor/act/ansible/roles/act_triage/`	Copies ACT to the host, runs it, reads its result.	No
`roles/service_watch/`	Checks the containers, records incidents, chooses approval or self-heal.	No
`playbooks/service_watch.yml`	**Job 1**: check the containers; propose (or apply) a fix.	No
`playbooks/service_fix_approved.yml`	**Job 2**: apply the fix a person approved.	No
`scripts/update-act.sh`	Replaces `vendor/act/` with a newer ACT release later.	No
`aap/credential_types/act_model_key.yml`	The credential type you paste into AAP (Part 5).	No

There is nothing to set: the containers are found by themselves (Part 2, "Which containers are watched"). You can add containers, leave some out, or go back to a fixed list (Part 5, step 4). Nothing is installed by hand on the hosts.

Part 2. The ACT script

What happens on the host during a run

1. The playbook finds the containers that should be running (root's and each user's) and checks each one with `podman container inspect`, as its owner. If all are running, the run ends here: ACT is not used and nothing is sent to GenAI.
2. If one is down, the playbook finds Python 3.8 or newer on the host.
3. It copies the ACT script to `/opt/act/act` (only when the file changed).
4. It runs ACT once, as root, with the GenAI key in ACT's environment (never on the command line, never printed). In approval mode the command looks like this:

```
/usr/bin/python3 /opt/act/act --non-interactive \
    --result-file /tmp/ansible.XXXXXX.act-result.json --max-steps 30 \
    'Health triage on host stigman01. These health checks failed:
     - container nginx is exited (exit code 0, stopped at 2026-09-25T09:06:02) ...'
```

In self-heal mode it may also start or restart the watched containers, and only the right way: `--allow='systemctl (start|restart|reset-failed) (stigman)(\.service)?'` for a container a systemd unit runs, `--allow='podman (start|restart)( (nginx))+'` for a plain one, and for a user's container the same, run as that user: `--allow='runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user (start|restart|reset-failed) (web)(\.service)?'`.

5. ACT asks GenAI what to look at and runs **read-only** commands: `podman ps -a`, `podman inspect nginx`, `podman logs --tail 80 nginx`, `journalctl -t podman ...`. It works out whether the container crashed, was stopped on purpose, or failed for another reason. **A user's container** (rootless): ACT refuses to run any command as another user in a job (`runuser -u ...`, `sudo -u ...`), so it cannot look at it itself. The playbook does it for ACT before step 4, read-only and as that user: the container's state, its last 60 log lines, `systemctl --user status` of its unit and the user's journal. It puts them in a file only root can read, ACT reads it with `cat`, and the playbook deletes it afterwards.
6. It writes its report (root cause, evidence, fix, confidence) and the fix it wants to run to a result file. The playbook reads it and deletes it.
7. The playbook checks the containers again itself. It does not take ACT's word for it.

What ACT can and cannot do

- **Never on the AAP server itself** (groups `aap`, `aap_hosts`, and the diagnose-only groups of `site_act_diagnose_only_groups`): there, self-heal is switched off whatever the survey says, and the apply job runs nothing - it prints the approved command for a person to run by hand. A restart there could stop the very job that is running (AAP 2.5 and later run in containers).
- Read-only commands (state, logs, disk space, open ports): yes, on its own.
- Approval mode: no changes at all. It only proposes, e.g. `systemctl start stigman.service`.
- Self-heal mode: start or restart of the watched containers, nothing else: `systemctl start` of the unit for a container a systemd unit runs, `podman start` for a plain one; for a user's container the same command run as that user (`runuser -u USER -- env XDG_RUNTIME_DIR=/run/user/UID ...`).
- Destructive commands (deleting, formatting, rebooting, changing users, firewall, SELinux): never on its own, whatever it is allowed.
- Everything ACT runs is listed in the job output and in the incident record.

What is sent to GenAI

The task text and the output of the read-only commands ACT runs (container state, container logs, journal lines). ACT removes passwords, tokens and keys first, then (ACT 0.6.18) replaces IP addresses, fully qualified names, the server's own names, local accounts and e-mail addresses with placeholders such as `host-1`, `domain-1.invalid`, `198.18.0.41` and `user-1`. The playbook also passes the server's inventory name to ACT, so `Health triage on host stigman01` reaches the model as `Health triage on host host-1`. ACT translates the model's answers back on the server before anything runs; the job output and the incident record show the real names. A short name ACT cannot recognize may still be sent. Make sure sending container logs to GenAI is allowed for your system.

Test ACT by hand first (10 minutes, on the STIG Manager host)

This proves the host can reach GenAI and the key works, before AAP is involved.

1. Copy the script from the repository to the host: `scp vendor/act/act you@stigman01:/tmp/act`
2. Log in to the host and check Python: `python3 --version` must show 3.8 or newer. (RHEL 8: `sudo dnf install -y python3.11`, then use `python3.11` below.)
3. Load the key without it showing on screen or in your history: `read -rs GENAI_KEY && export GENAI_KEY` (paste the key, press Enter).

4. Ask ACT something harmless: `python3 /tmp/act --non-interactive "Is sshd running? Answer in one line."` It should run a read-only command such as `systemctl show sshd ...` and answer.
5. Clean up: `unset GENAI_KEY` and `rm /tmp/act`.

If step 4 cannot connect, fix that first (firewall or proxy to `api.genai.mil`); AAP will hit the same problem.

Which containers are watched (same host)

The watch finds them by itself (`watch_discover: true`, the default): every container that **should be running**, whoever owns it:

- **Root's containers** (rootful podman, `sudo podman ps -a`).
- **Each user's containers** (rootless podman: only that user sees them). A user is looked at when a podman process runs as them, when they have *linger* on (`ls /var/lib/systemd/linger/`), or when they are in `/etc/subuid` and have container storage in their home. Their containers are read **as that user**, and only when `/run/user/<uid>` exists (otherwise podman would have to create it, and a check must not change anything).
- "Should be running" means: a systemd unit runs it and starts it at boot (a Quadlet file with an `[Install]` section, or an enabled unit), or its restart policy is `always` / `unless-stopped`. A container that exited with code 0 and has neither was stopped on purpose, and is not watched.

Each container is named with its owner in everything the watch prints: `nginx (root)`, `web (user alice)`. Root and a user (or two users) can each have a container called `nginx`; they are different containers and each gets its own fix.

A user who has units that should start at boot but whose systemd is not running (no `/run/user/<uid>`: linger is off and nobody is logged in as them) cannot be looked at; none of their containers run. The watch reports them as one entry, `the containers of user carol (1 should start at boot)`, with the fix `loginctl enable-linger carol` for a person to approve (self-heal never runs it: it changes how the user's session works, it does not just start a container). Every run also lists the users it could not look at (`Not looked at: user ...`).

The watch finds out by itself how each one is run: the container's `PODMAN_SYSTEMD_UNIT` label, a Quadlet file (`/etc/containers/systemd/stigman.container` for root, `~/.config/containers/systemd/` for a user), or a unit `NAME.service` / `container-NAME.service` that runs podman. A container a unit runs is started with `systemctl start <unit>`, never `podman start`: while the unit is stopped, Quadlet has usually removed the container, so there is nothing for podman to start. **A user's container is started as that user**:

```
runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user start web.service
runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 podman start web
```

(`runuser` runs one command as that user; `XDG_RUNTIME_DIR` tells podman and systemd where that user's session is. This form works on RHEL 8 and RHEL 9.)

To see what it finds, before AAP is involved:

1. `sudo podman ps -a --format '{{.Names}}'` lists root's containers; `ls /var/lib/systemd/linger/` the users whose containers keep running when they log out; `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/$(id -u alice) podman ps -a` one user's.
2. Run the **Health check** with only `containers` (`health_checks: containers`): its output has a table of every container it found, with its owner, state, unit and whether it should run.
3. The watch job prints the same in the task `Check | how the containers are run`.

4. In the test, stop a container the way it really stops: `sudo systemctl stop stigman.service` for a unit, `sudo podman stop nginx` for a plain container, and for a user's unit `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user stop web.service`.

Part 3. The playbooks

Job 1, `playbooks/service_watch.yml` (the whole file):

YAML

```
- name: Service watch
  hosts: "{{ target | default('stigman') }}"
  become: true
  gather_facts: false
  tasks:
    - name: Check, record, and self-heal or hand off for approval
      ansible.builtin.include_role:
        name: service_watch
        tasks_from: watch.yml
```

What it does, in order:

1. Checks that every watched container is running (and their pages answer, if you gave URLs).
2. All up: ends, green.
3. One down: records it, runs ACT (Part 2), checks again.
4. Approval mode: ACT's report is printed (task `Watch | ACT's report`), the record says `awaiting_approval`, and the job **fails on purpose** with `NEEDS APPROVAL ... Fix to approve (ACT): systemctl start stigman.service`. That failure is what sends the workflow to the approval step. The fix is made right before it is shown: a `podman start` ACT proposes for a container a unit runs becomes `systemctl start <unit>`; a command for a user's container is written the one way that works (`runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user start web.service`), whichever way ACT wrote it (`sudo -u alice ...`, `systemctl --user -M alice@ ...`); a down container ACT's fix does not cover gets the standard fix added; and when ACT proposes nothing (or cannot run: no key, no network), the standard fix is proposed, labelled `the standard fix (ACT proposed nothing)`. The commands are in dependency order. Example with a user's container down: `NEEDS APPROVAL on stigman01: container web (user alice) is not running: its systemd user unit web.service is failed ... Fix to approve (the standard fix (ACT proposed nothing)): runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user start web.service`.
5. Self-heal mode: if ACT's start worked and the re-check confirms it, the record says `self_healed` and the job ends **green**. Still down after ACT: the playbook runs the standard fix itself (`systemctl start <unit>` / `podman start <name>`) and checks again.
6. Still down and nothing to propose: fails with `STILL DOWN: ...` (a person must look).

Job 2, `playbooks/service_fix_approved.yml` (same shape, `tasks_from: apply.yml`):

1. Receives Job 1's result from AAP (`service_watch_fix`): the exact commands that were shown for approval. It refuses to run without it, so launching it on its own changes nothing.
2. If the container already came back, it changes nothing.
3. Otherwise it runs **exactly the approved commands itself**, in order, and stops at the first one that fails. No model, no key: nothing can be reworded between approval and fix. Each command's output and exit

code is in the job output (task `Fix | output`).

4. Checks again. Up: green. Still down: red.

Where incidents are recorded (one line per event):

- on the host in `/var/log/service-watch/incidents.jsonl`;
- in syslog with the tag `service-watch` (`sudo journalctl -t service-watch`);
- in each job's output in AAP (the task `Record | written to ...`).

Part 4. Put the code where AAP can reach it

1. Get the `site-automation` release tarball (or a copy of the repository).
2. Push it to a Git repository at work that AAP can reach (GitLab, Bitbucket, GitHub Enterprise).
3. Note the two values the AAP project needs (Part 5, step 3). AAP does not read them from your copy of the repository; you type them into the project once. In your copy, run:

```
git remote get-url origin     # Source control URL, e.g.
                              # https://gitlab.example.mil/ops/site-automation.git
git branch --show-current     # Source control branch/tag/commit when you use a branch, e.g. main
git tag --list                # the release tags, if you pin the project to one (e.g.
                              # `v<version>`)
```

Both are also in that copy's `.git/config`. If the repository is private, have a token or deploy key ready for the Source Control credential.

Part 5. AAP setup

Step 1. Credential type "ACT model key"

Skip this if it already exists from the ACT runbook.

1. **Automation Execution › Infrastructure › Credential Types › Create credential type**.
2. **Name**: `ACT model key`.
3. **Input configuration**:

YAML

```
fields:
  - id: genai_key
    type: string
    label: GenAI API key
    secret: true
  - id: asksage_key
    type: string
    label: AskSage API key
    secret: true
```

4. **Injector configuration**:

```yaml
env:
  GENAI_KEY: "{{ genai_key }}"
  ASKSAGE_API_KEY: "{{ asksage_key }}"
```

5. Click **Create credential type**.

Step 2. Credentials

Automation Execution › Infrastructure › Credentials › Create credential, three times:

Name	Credential type	Fill in
Linux ssh (sudo)	Machine	**Username**, **SSH Private Key**, **Privilege Escalation Method** `sudo` (plus the sudo password if needed)
ACT model key	ACT model key	the GenAI key in **GenAI API key**; leave AskSage empty
site-automation git	Source Control	only for a private repository: username + token, or SSH key

This is the only place the GenAI key goes. Nobody can read it back, and it never appears in job output, Git or the inventory.

Step 3. Project

1. **Automation Execution › Projects › Create project**.
2. **Name** `site-automation`; **Organization** yours; **Execution environment** `ee-minimal`; **Source control type** `Git`; **Source control URL** from Part 4; **Source control branch/tag/commit** `main` (a release tag later); **Source control credential** `site-automation git` if private.
3. Click **Create project**. Wait for **Successful**.

Step 4. Inventory

1. **Automation Execution › Infrastructure › Inventories › Create inventory › Create inventory**. **Name** `Linux servers`. Click **Create inventory**.
2. **Groups** tab › **Create group** › **Name** `stigman`. **Variables**: nothing is needed - the containers are found by themselves (Part 2, "Which containers are watched"). Optional, in the group's **Variables** (or in `playbooks/group_vars/stigman.yml`, where the release already lists the STIG Manager containers):

```yaml
watch_containers:            # watched in any case, in this order (dependencies first)
  - name: stigman
    url: https://127.0.0.1/  # also check that this page answers
  - name: nginx
  - name: web                # a container of a user (rootless podman)
    user: alice
podman_discover_ignore: ['test-.*', 'alice/scratch']   # never watch these (a name, or OWNER/NAME)
```

`unit:` under a container sets its systemd unit by hand, if the job output shows the wrong one. The containers in `watch_containers` are watched first, in their order, then the ones found. **To watch only your list** (the behaviour before 0.6.0), add `watch_discover: false`.

3. **Hosts** tab › **Create host** › the STIG Manager host's name. Then open the `stigman` group › **Hosts** tab › **Add existing host** › select it.

Step 5. Job templates

Automation Execution › Templates › Create template › Create job template, twice. Both use: **Job type** Run, **Inventory** `Linux servers`, **Project** `site-automation`, **Execution environment** `ee-minimal`, **Limit** empty with **Prompt on launch** ticked, **Credentials** `Linux ssh (sudo)`, plus `ACT model key` on *Service watch - check* (the apply job does not call the model). Do not tick **Prompt on launch** for **Variables** on the apply template: it runs the commands it is handed.

Name	Playbook
Service watch - check	`playbooks/service_watch.yml`
Service watch - apply approved fix	`playbooks/service_fix_approved.yml`

Part 6. The workflow

1. **Automation Execution › Templates › Create template › Create workflow job template**.
2. **Name** `Service watch - approve or self-heal`; **Inventory** `Linux servers`; **Limit** empty with **Prompt on launch** ticked. Leave **Variables** empty.
3. Click **Create workflow job template**. The workflow visualizer opens.
4. Click **Add step** › **Node type** *Job Template* › **Service watch - check** › **Finish**.
5. Hover over that box › **Add step and link** › **Node type** *Approval*:
   - **Name** `Approve the fix ACT proposed?`
   - **Description** `Read ACT's report and the red NEEDS APPROVAL line in the check job first.`
   - **Timeout** 30 minutes
   - Link type **Run on fail** (important: not "Run on success")
   - **Finish**.

If you schedule this workflow (for example every 15 minutes): open the workflow › **Edit** and turn **off** "Enable concurrent jobs" (it is off by default in AAP; check it). Then a run that is waiting for an approval (30 minutes here) is not joined by a second run every 15 minutes for the same containers.

6. Hover over the approval box › **Add step and link** › **Node type** *Job Template* › **Service watch - apply approved fix** › link type **Run on success** › **Finish**.
7. Click **Save**. (**Close** without **Save** throws the graph away.)
8. **Survey** tab › **Create survey question**: **Question** `When a container is down, what should happen?`; **Answer variable name** `watch_mode`; **Answer type** *Multiple Choice (single select)*; choices `approval` and `self-heal`; **Default answer** `approval`. Click **Create question**, then turn the survey **on**.

Why "Run on fail": AAP can only branch on success or failure. Job 1 fails on purpose only when a fix is waiting for a person, so an approval request appears only then.

Part 7. Approving a fix

Who can approve

Anyone who can run the workflow, and organization administrators. To let specific people approve: open the workflow template › **User Access** (or **Team Access**) › **Add roles** › **Approve**.

How to approve

1. The workflow stops at the approval box. The **Service watch - check** box is red. That is expected: red here means "a person is needed".
2. Open the **Service watch - check** job and read:
   - the task `Watch | ACT's report (root cause, evidence, fix)`: why it is down, the log lines that show it, the fix, and ACT's confidence;
   - the red line `NEEDS APPROVAL ... Fix to approve (...): <command>`: this is **exactly** what will run.
3. Go to **Automation Execution › Administration › Workflow Approvals** (or open the workflow job and click the paused box).
4. Select the request and click **Approve** or **Deny**.

Rehearse first: approve without changing anything

On *Service watch - apply approved fix* tick **Prompt on launch** next to **Job type**. In the workflow visualizer click the apply box › **Edit** › **Job type** `Check` › **Save**. Approve as usual: the apply job prints `DRY RUN on <host>: approved, and would run: systemctl start stigman.service. Nothing was changed (Check mode).` and ends green. Set the node back to `Run` when you trust it.

What happens next

- **Approve**: Job 2 checks the container, runs only the approved command, and checks again. Green means it is running. The record shows `fix_approved` then `fixed_after_approval`, and AAP shows who approved and when.
- **Deny** (or nobody answers within the timeout): the workflow stops and nothing is changed.

When to deny

- The check job says `STILL DOWN` instead of `NEEDS APPROVAL`: there is nothing safe to apply. Deny, and fix it by hand using ACT's report.
- The proposed command is not what you expect, or touches something other than the containers.

Part 8. The test: stop a container, approve the fix

1. In AAP, launch **Service watch - check**. Expected: `All watched containers are up (stigman (root), nginx (root)). Nothing to do.` - every container found is listed, with its owner.
2. On the host: `sudo podman stop nginx` (or `sudo systemctl stop <unit>` if systemd manages it). Check with `sudo podman ps -a`: nginx shows `Exited`.
3. In AAP, launch the workflow **Service watch - approve or self-heal**, answer `approval`.
4. The check box turns red. Open the check job and read `Watch | ACT's report`. A good report looks like this (the wording will differ):

```
ROOT CAUSE: the nginx container was stopped cleanly (exit code 0 at 09:06); its log ends
with a normal shutdown and there is no error before it. It did not crash.
EVIDENCE: podman inspect nginx: "Status": "exited", "ExitCode": 0; podman logs: "signal 3
(SIGQUIT) received, shutting down"
FIX: podman start nginx (not applied: needs approval)
CONFIDENCE: high
```

5. Approve it (Part 7). The apply box turns green; `sudo podman ps` shows nginx `Up`.
6. See the record on the host: `sudo tail -n 4 /var/log/service-watch/incidents.jsonl` (or `sudo journalctl -t service-watch -n 4`).
7. Repeat with `sudo podman stop stigman`. ACT should report that stigman stopped (it may also note that nginx can no longer reach it) and propose `systemctl start stigman.service` (or `podman start stigman`, if no unit runs it).
8. Optional: stop one again and run the workflow with `self-heal`. The check box goes green on its own; the record says `self_healed`.
9. Optional, a user's container (rootless): stop it as that user, e.g. `sudo runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user stop web.service`, and run the workflow with `approval`. The fix to approve is `runuser -u alice -- env XDG_RUNTIME_DIR=/run/user/1001 systemctl --user start web.service`, and ACT's report quotes the evidence the playbook collected for it.

Release 0.6.0 (finding the containers, rootless containers) was tested in nested UBI 9 (podman 5.8) and UBI 8 (podman 4.9, systemd 239) containers running systemd: root's Quadlet unit and plain containers, a user with linger and a Quadlet user unit stopped by hand, a user without linger: check job, approval fix (dry run and real) and self-heal; ACT was a scripted stand-in, and AAP was not involved. Last tested by hand at release 0.3.4 (check `CHANGELOG.md` for what changed since): every step on a RHEL 9 test machine running systemd and rootful podman, with Quadlet units (with `ContainerName=`, and the default name `systemd-NAME`), a hand-written unit that runs `podman run`, a plain container, and an `nginx.service` that is not the container (correctly ignored); ACT was a scripted stand-in. Not tested yet: your STIG Manager host, the real GenAI / Ask Sage, and AAP handing the check job's result through the approval step. This test is the first run with those. In step 5, the apply job's task `Apply | what was approved for this host` must list the command. If the apply job instead says `Nothing to apply: no results from the check job reached this job`, see Part 9.

Part 9. If something goes wrong

What you see	What to do
`ACT needs Python 3.8+`	RHEL 8: `sudo dnf install -y python3.11` on the host
`ACT produced no result` or `status: error`	the host cannot reach GenAI or the key is wrong: repeat Part 2's hand test
`ASYNC FAILED` on the ACT task	normal when ACT is waiting for approval; read the next tasks
No approval step appears	the link from the check box must be **Run on fail**
Check job says `STILL DOWN`	nothing to approve: deny, fix it by hand
`container nginx does not exist`	the name is different: fix `watch_containers` (Part 5, step 4)
`container web (user alice) cannot be checked: there is no /run/user/1001`	that user's systemd is not running (linger is off and nobody is logged in as them), so none of their containers run. The fix proposed is `loginctl enable-linger alice` (their containers then start at boot and keep running after logout)
A container is watched that you do not want watched	leave it out: `podman_discover_ignore: ['its-name']` (or `['alice/.*']` for all of one user's)
You want the old behaviour: only your list	`watch_discover: false` (then only `watch_containers` is watched)
`... is in aap (diagnose only): self-heal is off here`	the host is the AAP server (or in a diagnose-only group): fixes wait for approval, and the apply job prints them for you to run by hand
`the containers of user carol (...) cannot be checked`	carol has units that should start at boot, but her systemd is not running. Approve `loginctl enable-linger carol` (or run it by hand); if her containers should not run, leave her out: `podman_discover_ignore: ['carol/.*']`
The container comes back by itself before the check runs	systemd restarts it: stop it with `sudo systemctl stop <unit>`
`Check | how the containers are run` names the wrong unit, or none	set it by hand: `unit: stigman.service` under that container in `watch_containers`
Apply job: `Nothing to apply: no results from the check job reached this job`	launched on its own (use the workflow), or Job 1 failed before ACT ran (read its output). If Job 1 did say `NEEDS APPROVAL`, AAP did not pass its result along: start the container by hand for now and report it